

VOID: build plan

How to get from a marketing site to a product people run in their write path. Written 3 September 2026. Assumes one developer with heavy model assistance, roughly 25 hours a week.

1. What you are actually building

A process that sits between an AI agent and the tools it can write to. For each call it decides a reversibility class from the state of the target, applies a policy, and appends a signed record. Later it can replay the inverses.

Everything else on the site is a consequence of that sentence. When a decision is unclear, the test is: does this make the write path more accountable, or is it a feature that happens to be nearby.

The honest gap between the site and the product

The site demonstrates the finished idea. Three parts of it are materially harder than they look, and the plan below sequences around that:

ON THE SITE	REALITY
Held calls return a provisional receipt and the agent keeps working	Needs speculative execution and a dependency graph. Version one holds the call blocking , which works with every agent today and needs nothing from the framework.
89 registry entries with compensations	Data, not code. Each real compensation is its own connector with its own auth, snapshot format and failure modes. Ship three.
Preconditions evaluated at intercept time	Requires probing the target, which requires credentials VOID does not have in proxy mode. See section 7.

None of these make the pitch dishonest. They make it a roadmap.

2. The one decision that determines everything else

Pick one agent host and one tool surface, and refuse to widen until both work end to end.

The recommended wedge:

- **Agent host:** Claude Code or any MCP capable client. It speaks MCP natively, it is used for work that touches real systems, and its users already feel the problem.
- **Tool surface:** Postgres, through an MCP server.

Why Postgres first, ahead of the flashier options:

1. The blast radius is computable exactly. A counting `SELECT` inside a transaction you abort gives you a real number, not an estimate.
2. Compensation is genuinely achievable. A before image plus a keyed update is a true inverse, not a mitigation.
3. `DDL` is transactional, so `ROLLBACK` is a working undo for the whole class.
4. You control both sides in a test environment, so you can prove an undo actually happened instead of asserting it.

Stripe is the better demo and the worse first build: nearly everything in it is R2 or R3, so you get a product that says no and never gets to say "undone". Build the thing that can prove itself first, then add the thing that sells.

3. Model setup, and what changed today

GPT-6 Astra shipped on 3 September 2026. The numbers that matter for this build:

Model id	<code>gpt-6-astra</code>
Context	1,050,000 tokens
Max output	128,000 tokens
Input	\$10 per million
Output	\$50 per million
Cached input	\$1 per million
Cache write	\$12.50 per million
Batch	50 percent of standard
Knowledge cutoff	30 April 2026
Tools	function calling, structured outputs, prompt caching, computer use, MCP, file search, web search, code interpreter

Access reality: it rolled out on launch day to enterprises in the Trusted Access Program, with API and Plus / Pro / Business / Enterprise access following "in the coming days". It is also on Amazon Bedrock. Plan for it, do not block on it, and keep the build model configurable so a fallback is a config change rather than a rewrite.

The five things that actually change how you work

- 1. Prompt caching is the entire cost strategy.** Cached input is a tenth of fresh input. But a cache write costs *more* than a plain input token, so the win only arrives on reuse. The discipline this implies:
- Build one **stable prefix** and never reorder it: architecture doc, registry schema, coding standards, the interfaces you have frozen.
 - Put everything volatile (the task, the current file, the error) **after** it.
 - One 200k token prefix costs \$2.50 to write, then \$0.20 per call instead of \$2.00. Over a hundred calls in a working session that is \$20 against \$200.
 - Reordering the prefix throws the cache away. Treat prefix stability as a rule, not a preference.
- 2. A million tokens removes most retrieval plumbing at this size.** The whole VOID codebase, the MCP specification and a vendor's API docs fit in one call together. Do not build a RAG pipeline for a repository this small. Feed it the files.
- 3. Computer use lets you verify instead of assert.** The failure mode of an undo feature is a passing test over a mocked client. Have the model drive the actual Stripe test dashboard or AWS console and confirm the row came back. This is the single highest value use of the new capability for this product.
- 4. Batch at half price for the registry.** Extending the registry from 89 to 500 entries is an embarrassingly parallel classification job with a fixed output schema. Structured outputs make each entry valid by construction. Run it as a batch job overnight for single digit dollars rather than as interactive work.
- 5. Route by difficulty, not by habit.** At \$10 in and \$50 out, using the top model for mechanical edits is waste. A rough split:

WORK	MODEL
Architecture, protocol design, hard debugging, security review	Astra
Feature implementation against a written spec	Mid tier
Renames, test scaffolding, docstrings, mechanical refactors	Cheapest available
Registry entry generation	Batch, mid tier, structured outputs

One hard safety rule

Astra is the first model OpenAI has placed at the **critical** cybersecurity threshold in its preparedness framework: it can find and exploit unknown vulnerabilities across defended systems without step by step guidance. VOID is a product that will hold credentials and sit in a privileged position in someone else's infrastructure.

Never point an autonomous agent with computer use at a production account during development. Test accounts, disposable projects, and a separate cloud account with a spend

cap. Write it down as a project rule on day one, because the moment you are tired and it would be convenient is the moment it costs you.

4. Repository shape

Fold the product into the existing workspace rather than starting a second repository. The registry already exists in `api/_registry.ts` and is read by three consumers; making it a package is what stops the site and the runtime from ever disagreeing about what a class means.

```
packages/  
  registry/      the data, the case evaluator, the JSON schema  
  ledger/        hash chain, signing, verification, export  
  proxy/         the MCP proxy: server side, client side, transports  
  policy/        match rules, decisions, approval channels  
  connectors/    one directory per tool surface, snapshot and inverse  
  cli/           void run, void ledger verify, void replay, void attest  
apps/  
  site/          the current marketing site  
  control-plane/ approvals, live feed, ledger browser
```

Rules that keep this from rotting:

- `packages/registry` imports nothing. Not the ledger, not node built ins. It is data and pure functions, which is why the browser, the API and the runtime can all read it.
- A connector never imports another connector.
- The proxy never imports a connector directly. It resolves them through a registry of connector ids, so adding a tool surface is additive.
- The ledger is append only in the type system, not just by convention. Expose `append` and `read`. Do not expose `update`.

5. Phases

Each phase has an exit criterion you can actually run. If you cannot run it, the phase is not finished, whatever the code looks like.

Phase 0: scaffold and decide (3 days)

Write a decision log before writing code. Five decisions, one paragraph each: the wedge, the storage, the signing key path, the policy format, the licence. You will change your mind about two of them and the log is what tells you why you chose the original.

Set up the harness first: typecheck, lint, test, one command. You already know from this repository what an unrunnable build costs.

Exit: `pnpm test` runs and passes with one trivial test.

Phase 1: transparent proxy (1 week)

The riskiest integration work, so it goes first. Build an MCP proxy that changes nothing about the session. It terminates the agent's connection as a server and opens its own to the upstream server as a client.

What has to work before you claim it is transparent:

- `initialize` and capability negotiation, forwarded honestly. Do not advertise capabilities the upstream does not have.
- `tools/list`, `tools/call`, and the resource and prompt equivalents.
- Notifications in both directions, including `notifications/tools/list_changed`.
- Progress tokens, so long running calls still stream.
- Cancellation.
- Both transports: stdio framing, and Streamable HTTP.
- Upstream reconnect without the agent noticing.

Exit: run a real half hour agent session twice, once direct and once through the proxy, and diff the transcripts. Any difference other than latency is a bug.

Traps: stdio framing is unforgiving about partial reads. Cancellation is the one everybody skips and it is the one that produces hung agents in a demo.

Phase 2: the ledger (4 days)

Every intercepted call appends one entry. The entry carries intent, agent and tool identity, timestamp, a digest of the payload rather than the payload, the class, the decision, the snapshot reference, and the link to the previous hash.

The sealing algorithm already exists in this repository in `api/_core.ts`. Lift it, do not reinvent it. Sign the head with ed25519 from an environment key in development and from KMS in production.

Exit: three commands pass.

```
void ledger verify          # 1000 real entries, chain intact
void ledger verify --tamper 42 # edit entry 42, body check catches it
void ledger verify --drop 42  # delete entry 42, link check catches it
```

Those two failures are different failures. If your verifier only walks links, an edited record passes. Rehash the body.

Phase 3: classification (1 week)

Turn the registry into a runtime evaluator. Input: a tool id, its arguments, and a context object describing the target. Output: the first case whose precondition holds, with the class, the inverse and the note.

Version one gets preconditions from **declared configuration**, not probes:

```
targets:
  - match: { connector: s3, bucket: prod-exports }
    facts: { versioning: enabled, mfa_delete: false }
    verified_at: 2026-09-03
```

This is not a compromise you should be embarrassed by. It is auditable, it is fast, and it forces the operator to state what they believe about their own infrastructure, which is itself valuable.

Probing replaces it in phase 6.

Exit: run an hour of real agent traffic and print the class distribution. If more than a fifth of calls come back unclassified, the registry has the wrong coverage for your wedge and you fix that before continuing.

Phase 4: policy and the blocking hold (1 week)

Start with matching rules, not a language. A language is a project of its own and you do not yet know what people need to express.

```
rules:
  - match: { class: r0 }
    decision: allow
  - match: { class: r1, blast_radius: { lt: 100 } }
    decision: allow
  - match: { class: [r2, r3] }
    decision: hold
    seconds: 120
    notify: [cli, slack]
  - match: { tool: "postgres.table.drop" }
    decision: deny
```

The hold **blocks the tool call**. The MCP response does not return until a human decides or the timer runs out. This works with every agent that exists today and requires nothing from the framework. On cancel, return a proper MCP error with a reason the model can read and act on, not a timeout.

Exit: an agent tries a real R3 write against a test account, you get a notification, you cancel it, the write never happens, and the agent recovers and says something sensible about why.

That moment is the product. Everything before it is plumbing and everything after it is scale.

Phase 5: compensation, three calls only (2 weeks)

Not eighty nine. Three:

1. `postgres.row.update` : before image via keyed `SELECT` , inverse is a keyed `UPDATE` .
2. `postgres.row.delete` : capture full rows plus anything reachable through `ON DELETE CASCADE` , inverse is an ordered `INSERT` .
3. `s3.object.delete` : check versioning, inverse is removing the delete marker.

The cascade case in number two is the one that teaches you the shape of the problem. One statement becomes writes across several tables, and the inverse has to run in dependency order or the foreign keys reject it.

Snapshots go to a bucket the operator nominates. VOID stores the reference and the digest, never the payload. Get this boundary right now, because changing it later means a migration through customer data.

Exit: make a real change through an agent, run `void replay --to <time>` , then open a database client and confirm with your own eyes that the row is back and identical. Not a test assertion. Your eyes, then a test.

Phase 6: probes and blast radius (1 week)

Now replace declared facts with measurement, and add the number that makes approvals meaningful.

- Postgres: run the predicate as `SELECT count(*)` inside a transaction you abort. Walk `information_schema` for cascades.
- S3: `ListObjectsV2` on the prefix, then a versioning probe per key.
- Stripe: read the object and check its current state.

Probes need a cache with a TTL and a clear staleness policy. A probe that runs per call will double your latency and your API bill.

Exit: the hold notification carries a real number, and that number is correct when you check it by hand.

Phase 7: control plane (2 to 3 weeks)

The smallest thing that makes VOID usable by someone who is not you:

- A live feed of intercepted calls with class and decision.
- An approval queue with the blast radius and the compensation plan visible.
- A ledger browser with verification status.
- Replay, with a preview of what will be compensated before you commit.

Do not build multi tenancy, billing or SSO yet. One organisation, one Postgres, an environment variable for the token.

Phase 8: taint graph (2 weeks)

Correlate reads to writes so a compensation knows its real scope. This is the part with no prior art, so budget for it going wrong twice.

The mechanism: every read is recorded with the identity of what it returned. Every write records which read results were in the agent's context when it was issued. That gives you edges.

Transitive closure over those edges gives you the scope.

The honest limitation, which you should state in the product rather than hide: you can prove a read *preceded* a write in the same context. You cannot prove it *caused* it. What you have is a superset of the true causal set, which is the correct direction to be wrong in for a safety tool.

Phase 9: attestation and export (1 week)

Period export, signed, verifiable by someone who does not trust you. The verifier is a hundred lines and it should be open source and separately installable. A verification tool that only runs inside the product it verifies is not evidence.

Phase 10: second deployment shape (2 weeks)

Only after the MCP proxy has a real user. The SDK wrap is the natural second because it reuses the whole spine and only changes the interception point.

6. Timeline

PHASE	EFFORT	CUMULATIVE
0 scaffold	3 days	3 days
1 transparent proxy	1 week	1.5 weeks
2 ledger	4 days	2 weeks
3 classification	1 week	3 weeks
4 policy and hold	1 week	4 weeks
5 compensation, three calls	2 weeks	6 weeks
6 probes and blast radius	1 week	7 weeks
7 control plane	2.5 weeks	10 weeks
8 taint graph	2 weeks	12 weeks
9 attestation	1 week	13 weeks
10 SDK wrap	2 weeks	15 weeks

Roughly four months to something a friendly first user can run, at 25 hours a week. Phase 4 is the demo. Phase 5 is the product. Everything after phase 7 is what turns a tool into a company.

Add 30 percent. Every estimate in this table is what the work costs when nothing surprises you, and phase 1 and phase 8 will both surprise you.

7. The three hard problems, named early

Every project has a small number of problems that decide whether it works. Find them now rather than in month three.

7.1 In proxy mode, VOID does not hold the credentials

This is the architectural problem at the centre of the product and the site does not show it.

In MCP proxy mode the upstream MCP server holds the tool credentials. VOID sees the calls but cannot independently ask S3 whether versioning is on, because it has no S3 credentials of its own. So the precondition it needs in order to classify is exactly the thing it cannot reach.

Three ways out, in the order you should try them:

1. **Declared facts (phase 3).** The operator states the facts in config and VOID records when they were last verified. Cheap, auditable, and honest about what it knows. Ships first.
2. **Probe through the upstream server.** Most MCP servers expose read tools next to their write tools. VOID calls the read tool to establish the fact before forwarding the write. Elegant, no new credentials, and it works whenever the server exposes the right read. Ships in phase 6.
3. **Its own read only credentials per connector.** Most accurate, and it moves VOID from a transparent proxy to a component with standing access, which is a much larger security and sales conversation. Offer it, do not require it.

Decide the order now. Building option 3 first is the mistake that turns a proxy into an agent platform.

7.2 The speculative hold needs a dependency graph

The blocking hold in phase 4 works everywhere and is genuinely useful. The provisional receipt version on the site is better product and much harder: once the agent takes a provisional id and issues three more calls against it, cancelling the first has to withdraw the other three.

You need, at minimum: provisional identifiers the agent can carry, a record of which provisional values appeared in the arguments of later calls, a cascade that withdraws transitively, and a story for what happens when a dependent call has already committed to a system that does not hold.

Do not attempt this before phase 8. It is the same machinery as the taint graph, and building it twice is how you end up with two half implementations.

7.3 Snapshots are the only place real data lands

Everything else VOID stores is metadata and digests. Before images are actual customer data, which means:

- Retention policy per connector, enforced, defaulting to short.
- Field level redaction before the snapshot leaves the process.
- The bucket belongs to the customer, never to you, and this is not negotiable even for the first user.
- Storage cost grows with write volume, and the pricing model has to account for that or a heavy user becomes unprofitable.
- Under GDPR a snapshot of personal data is processing, and a deletion request has to reach it.

Get the boundary right in phase 5. Changing where snapshots live after you have users means migrating their data, which is the worst kind of work.

8. What not to build

The list matters more than the roadmap, because every item here is a plausible good idea that would cost you a month.

- **All 89 registry connectors.** Three real ones beat eighty nine stubs. The registry as *data* can grow cheaply through batch generation; the registry as *implementations* grows at one per week and only when a user asks.
- **A policy DSL.** Matching rules until somebody hits their limits and tells you what they needed.
- **Multi tenancy, billing, SSO.** Not before the second paying user.
- **A dashboard before the CLI.** The CLI is how you will debug the proxy for the next year. Build it first and keep it good.
- **Your own agent framework.** VOID is valuable precisely because it is framework agnostic. The moment it has opinions about the agent loop, it becomes a competitor to the thing it should plug into.
- **The taint graph before compensation works.** Scope is only interesting once you can act on it.
- **Support for every MCP transport.** stdio and Streamable HTTP. That is all.

9. Cost model

Development model spend, assuming caching discipline and roughly 30 calls on a working day with a 150k token stable prefix:

ITEM	ARITHMETIC	PER DAY
Cached prefix reads	$30 \times 150k \times \$1/M$	\$4.50
Fresh input	$30 \times 20k \times \$10/M$	\$6.00
Output	$30 \times 8k \times \$50/M$	\$12.00
Cache writes	$4 \times 150k \times \$12.50/M$	\$7.50
Total		\$30.00

About **\$630 a month** at 21 working days.

The same work with no caching, sending the prefix fresh every call:

ITEM	ARITHMETIC	PER DAY
Fresh prefix	$30 \times 150k \times \$10/M$	\$45.00
Fresh input	$30 \times 20k \times \$10/M$	\$6.00
Output	$30 \times 8k \times \$50/M$	\$12.00
Total		\$63.00

About **\$1,320 a month**. Caching discipline is worth roughly \$700 a month, which is more than the rest of the infrastructure combined. It is a prefix ordering rule, and it pays for the hosting.

Everything else:

ITEM	MONTHLY
Postgres, managed, small	\$25
Hosting, control plane and site	\$20
Object storage for test snapshots	\$5
Domain	\$2
Infrastructure total	\$52

Realistic burn: \$500 to \$900 a month, dominated by model spend, and roughly halved on weeks where you are implementing to a written spec rather than designing.

10. Risk register

RISK	LIKELIHOOD	IMPACT	WHAT TO DO ABOUT IT
Agent frameworks ship native approval hooks	High	High	This commoditises the hold, not the registry or the ledger. Keep the registry the centre of gravity and treat the hold as a feature, not the moat.
Rubrik or another incumbent extends into vendor neutral interception	Medium	High	They are bound to their own data plane. Stay on the calls they structurally cannot reach: external sends, payments, third party SaaS.
MCP specification changes under you	Medium	Medium	Keep the transport layer thin and behind an interface. Do not spread protocol details through the codebase.
The probe credential problem has no clean answer	Medium	High	Declared facts already ship a working product. Treat probing as an upgrade, not a dependency.
Snapshot storage becomes a data liability	Medium	High	Customer bucket, short retention, redaction before the snapshot leaves the process. Decided in phase 5, not later.
Astra access is delayed past your build window	Medium	Low	Keep the model behind config. Nothing in the plan requires it specifically.
Solo bandwidth, four months is long	High	High	Phase 4 is a demo you can show people. Get in front of five potential users at week four rather than at week fifteen.

11. The first week, concretely

Day 1. Decision log, five decisions. Workspace restructure: move `api/_registry.ts` into `packages/registry` and repoint the site at it. Prove nothing broke. Set up the test harness.

Day 2. Read the MCP specification properly, both transports, end to end. Not skimmed. This is the protocol your product lives inside and a day here saves a week in phase 1. Write down every message type you will have to forward.

Day 3 and 4. Stdio proxy that forwards `initialize`, `tools/list` and `tools/call` and nothing else. Get one real agent talking to one real MCP server through it.

Day 5. Notifications, progress and cancellation. This is the day that will overrun.

Weekend, optional. Streamable HTTP transport.

End of week exit: a real agent session runs through the proxy and the transcript is identical to running direct. Nothing is classified, nothing is held, nothing is logged. Just a pipe that does not leak.

That is the correct week one. It looks like no progress and it is the foundation everything else stands on.

12. Why the timing is better today than it was yesterday

GPT-6 Astra shipped this morning with autonomous computer use, and OpenAI classified it as the first model to reach the **critical** threshold on cybersecurity in their own preparedness framework. It is going to enterprises first, through a trusted access programme, precisely because of what it can do unsupervised.

Read that as a product signal rather than as news. The industry is now shipping models that act on real systems without step by step human guidance, and the people deploying them first are the ones with the most to lose. A layer that answers "what did it just do, can I take it back, and can I prove either" is worth more this week than it was last week.

That does not make you first. Rubrik has been in this positioning for a year and the MCP gateway category is crowded. It makes the question the market is asking louder, and the specific answer VOID gives, which is that reversibility is a property of a call evaluated against the state of its target, is still nobody else's answer.

Build phase 4. Show it to five people who run agents against production. Their reaction tells you whether to build phase 5 or a different phase 5.

Sources

- [GPT-6 Astra model reference, OpenAI](#)
- [OpenAI API pricing](#)
- [OpenAI begins rolling out Astra after warning of its advanced cyber capabilities, CNBC](#)
- [OpenAI debuts GPT-6 Astra, says it triggered security measures, NBC News](#)
- [Path to Astra: critical capabilities and frontier safeguards, OpenAI](#)
- [OpenAI launches GPT-6 Astra, touts its ability to use your computer, Fortune](#)